Tobacco Leaf Disease Diagnosis

Adaptive Federated Learning with Client Selection and Gradient Compression

B.Tech project report: design, progress, results, limitations and future scope

Report date	1 November 2026
Project status	Web application and baseline classifier complete and tested locally. The federated-learning experiments (the core research question) are built but **not yet run**. Nothing is deployed.
Headline result	Centralised baseline: **84.1% accuracy**, macro-F1 0.729 on 138 held-out photos (16 classes). Roughly +/-6 points of uncertainty.

How to read this report: sections 1 and 2 explain the project and its status in plain language. Sections 3 to 8 describe how it is built. Sections 9 to 12 give the verification, the results and their limits. Sections 13 and 14 list what remains. A glossary in the appendix explains technical terms.

Contents

1. Executive summary 3

2. Background and motivation 4

3. Dataset 5

4. System overview 6

5. Machine-learning design 7

6. Web application 9

7. Prediction service 10

8. Verification and testing 11

9. Results: the centralised baseline 12

10. Reliability: what the numbers can and cannot support 15

11. Limitations 15

12. Completion status 16

13. Future scope and remaining work 16

14. How to run and reproduce 17

15. References and credits 18

Appendix A. Glossary 18

Appendix B. The 16 classes in plain language 19

1. Executive summary

This project builds and studies a system that helps identify diseases on tobacco leaves from a photograph. It has two parts that are kept separate on purpose: a **web application** that people use, and a **machine-learning pipeline** in Python that trains and tests the models.

The research question is about **federated learning**. Normally, training a model means collecting all photos in one place. In federated learning the photos stay with their owners (here, simulated farms or labs) and only model updates travel to a central server. That protects data, but it costs network traffic and is complicated by the fact that different clients hold very different kinds of photos. The project tests two ideas that may reduce those costs: **adaptive client selection** (choose which clients train in each round using a score instead of luck) and **gradient compression** (send only the largest changes, called Top-k). Four experiments, A to D, compare every combination.

Where the project stands. The website, the sign-in and data-security rules, the federated-learning code, a results import tool and a prediction service are built and tested. A conventional (non-federated) baseline classifier has been trained and evaluated. The four federated experiments have not been run on real data yet, so the project's central question is still unanswered. Nothing has been deployed or committed to version control.

Status at a glance

Part of the project	Status	Evidence
Dataset audit and preparation	Done	16 classes, leakage controls, synthetic images excluded from evaluation
Web application (9 pages)	Done	Runs locally; lint and type checks pass
Sign-in and database security rules	Done	10 automated rule tests pass on the Firebase emulator
Federated-learning code (FedAvg, selection, Top-k)	Done	20 Python tests pass; tiny smoke runs only
Results import into Firebase	Done	Round-trip test passes with smoke-test data
Centralised baseline classifier	Done	84.1% accuracy, macro-F1 0.729 (test set, 138 photos)
Prediction service and diagnosis page	Done	Works locally with real uploads; runs on one PC only
Federated experiments A to D on real data	Not started	Need a GPU session; the main missing piece
Real federated results shown in the web app	Not started	Depends on the experiments; charts only seen with smoke data
Check on photos from other sources	Not started	Three informal spot checks all failed (section 11)
Deployment (Vercel) and hosting of the service	Not started	Needs the owner's approval
Final report with real federated results	Partly	This report covers everything up to the baseline

Completion in one line. 7 of 12 work items are done (this is a simple count, not a weighted measure of effort). The largest remaining item, the federated experiments, is also the most important one scientifically.

2. Background and motivation

Why tobacco leaf diseases matter. Tobacco is vulnerable to fungi, bacteria, viruses, insect pests, nematodes (microscopic soil worms) and environmental damage. Many of them show up first as spots, patches or colour changes on leaves. Recognising them early helps growers act sooner, but identifying a disease by eye takes experience and is easy to get wrong because several diseases look alike.

Why use a computer-vision model. A neural network can learn to recognise patterns in leaf photographs. The model used here, **MobileNetV2**, is small and fast, which makes it suitable for phones and simple servers. It is first trained on a large general photo collection (ImageNet) and then fine-tuned on tobacco leaves, a standard approach when there are only a few thousand training photos.

Why federated learning. Photos of crops may belong to different farms, cooperatives or research stations that do not want to hand over raw data. In federated learning, each client trains on its own photos and sends only the change to the model. The server combines those changes (the classic method is *FedAvg*, a weighted average) into a better shared model. Two practical problems follow:

- **Communication cost.** Every round, every selected client downloads and uploads a full model. Networks in rural areas can be slow or expensive.
- **Uneven data (non-IID).** One farm may see mostly one disease. Updates from such clients pull the shared model in different directions, which can slow learning.

The two ideas under test.

- **Adaptive client selection.** Instead of picking clients at random each round, give each client a score and pick the highest. In this project the score combines how badly the client did recently (50%), how long it has been waiting (30%) and how much data it holds (20%).
- **Gradient compression (Top-k).** After local training, a client sends only the largest 10% of its changes, plus their positions. The part it did not send is remembered and added to the next update (called error feedback), so little information is permanently lost.

What the project hopes to show. Whether either idea, or both, reduces total network traffic or the number of rounds needed, without losing accuracy compared with plain FedAvg. The outcome could be positive, neutral or negative; the experiments have not been run, so no conclusion is claimed in this report.

Objectives

- Build a web application where signed-in users can upload a leaf photo, receive a clearly labelled model estimate and read results of experiments.
- Implement a federated-learning pipeline with FedAvg, random and adaptive client selection, and Top-k compression.
- Run four controlled experiments (A to D) that differ only in selection and compression, and compare accuracy, convergence speed and communication cost.
- Report results honestly, including limits, small-sample effects and anything that was not verified.

3. Dataset

The project uses the **Tobacco Leaf Abnormality (TLA)** dataset, published on Hugging Face as `TamAko783/TPDD_Honglin_CLS`. It has 16 classes and 1,571 images in total. The images were colour-normalised by the dataset authors with the Shades-of-Gray method.

Real versus synthetic images. Part of the training split consists of synthetic copies (images with `_aug` or `_paste` in their names). This project trains on real photographs only, and the code asserts that no synthetic image can appear in the validation or test split. That leaves 1,122 training, 138 validation and 138 test photos (1,398 real photos in total; 173 synthetic training images are excluded).

Leakage controls. The dataset's own leakage-aware split is kept. Even so, four classes have a single validation photo and a single test photo, so the scores of those classes are extremely sensitive to one image (section 10).

Real photos per class and split

Class	Train	Val	Test	Synthetic (not used)
anthracnose	33	1	1	27
black shank	7	1	1	53
brown spot	80	10	10	0
cmv	82	10	10	0
frog eye	80	10	10	0
genetic abnormality	17	1	1	43
healthy	80	10	10	0
nematodes	66	8	8	0
potato tuber moth	80	10	10	0
pvy	52	8	8	8
sunscald	108	14	14	0
target spot	80	10	10	0
tmv	103	14	14	0
tswv	18	1	1	42
weather fleck	94	12	12	0
wildfire	142	18	18	0
Total	**1122**	**138**	**138**	**173**

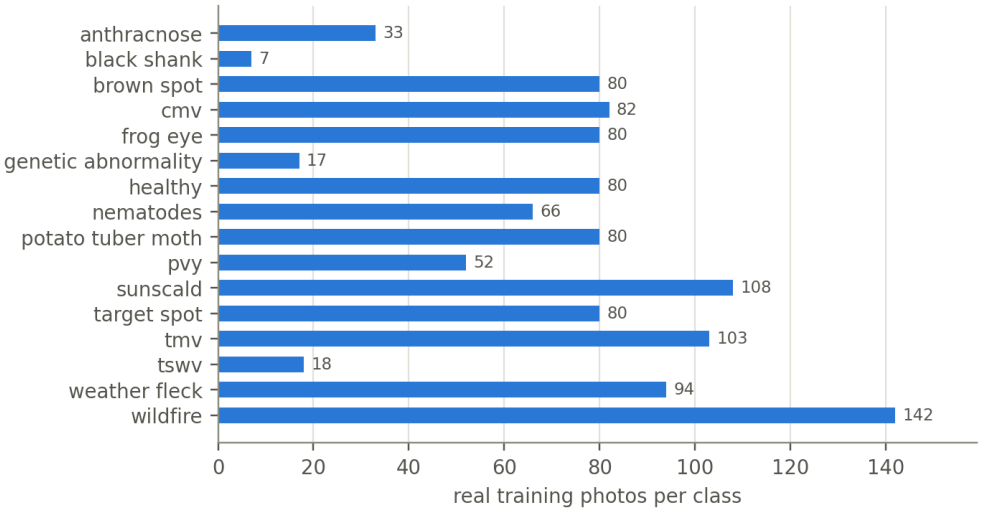


Figure 1. Real training photos per class. Classes are strongly unbalanced: wildfire has 142 photos, black shank has 7.

Licence and credit. The source repository states no explicit licence. It asks for academic use only, citation of the source papers (Lin et al., SPIE ICDIP 2022; Frontiers in Plant Science 2024) and no redistribution. For that reason the dataset is not stored in the project repository, in Firebase or on the hosting platform; it is downloaded on demand.

Datasets considered and not used. An earlier Kaggle tobacco dataset was a binary segmentation set, not a disease-classification set, and was dropped. A Roboflow classification dataset (CC BY 4.0, 1,268 images) has only three classes: *alternaria alternata* (the cause of brown spot), *cercospora nicotianae* (the cause of frogeye leaf spot) and a third class meaning neither of those is present. That third class is not the same as healthy, so merging it into the 16 classes would create wrong labels. It may be useful later as an external check for those two diseases.

4. System overview

The system has a strict separation: **training happens offline in Python**. The website never trains anything; it displays results that were imported, and it sends a photo to a prediction service when a user asks for a diagnosis.

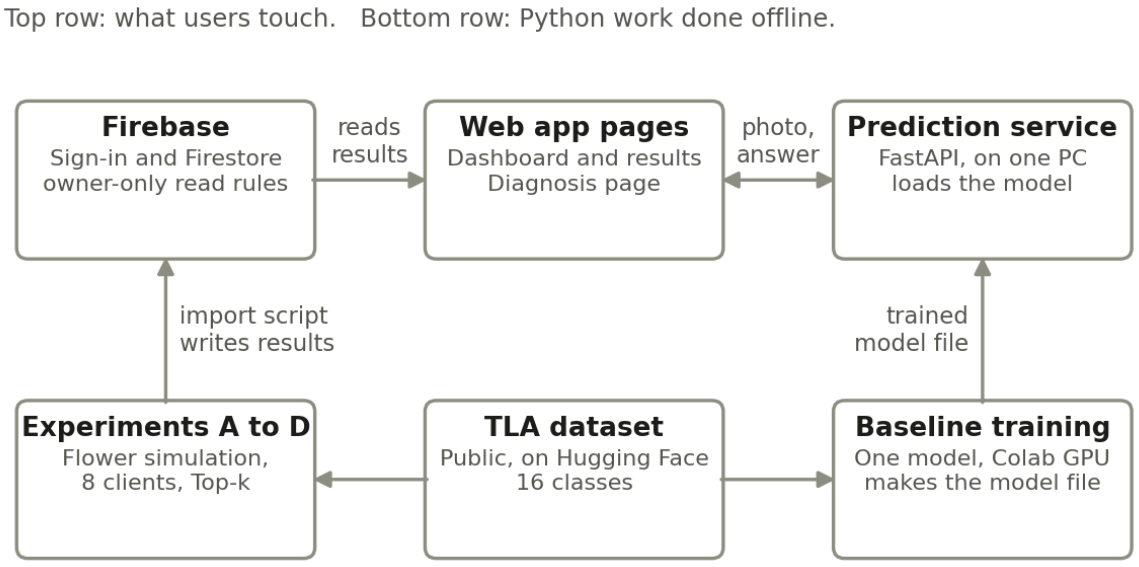


Figure 2. System components. The top row is what users touch; the bottom row is offline Python work.

Technology used

Layer	Tools
Website	Next.js (App Router), React, TypeScript, Tailwind CSS, Recharts for charts
Accounts and data	Firebase Authentication (email and password) and Firestore, protected by security rules
Model	PyTorch, MobileNetV2 pre-trained on ImageNet, fine-tuned on TLA
Federated learning	Flower 1.42 (simulation mode with the Ray backend), FedAvg, custom strategy
Prediction service	FastAPI and uvicorn, loads a saved model file, no image is stored
Testing	pytest, the Firebase emulator with @firebase/rules-unit-testing, ESLint and TypeScript checks
Training hardware	Developer laptop (4 CPU cores, no GPU) for development; Google Colab GPU for the baseline

5. Machine-learning design

5.1 Model and local training

Every client and the central baseline use the same **MobileNetV2** network with a 16-way output layer. Local training uses stochastic gradient descent with momentum 0.9 and weight decay 1e-4, with random horizontal and vertical flips and small brightness changes as augmentation. The model's weights, including its BatchNorm statistics, are exchanged as one flat list of 32-bit numbers. A client's *update* is the weights after local training minus the weights it received.

5.2 One federated round

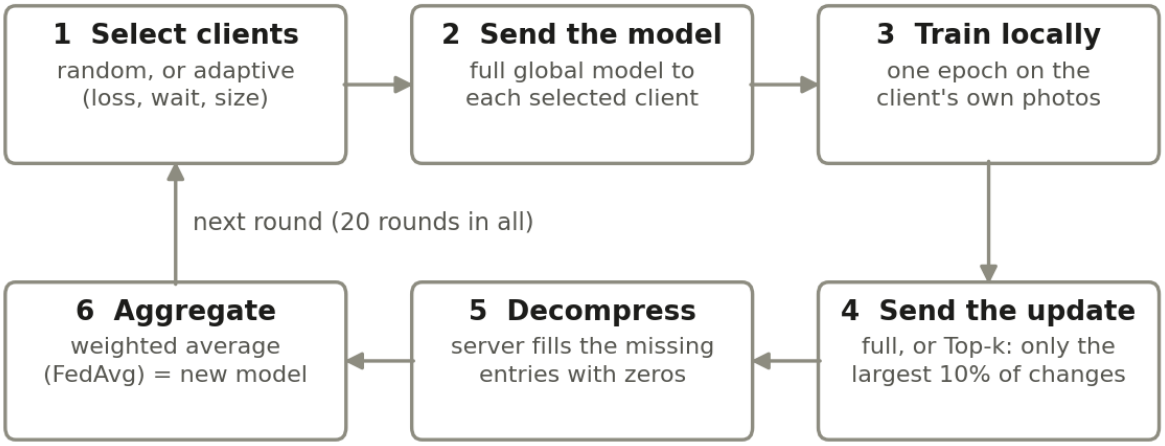


Figure 3. One federated round. The full cycle repeats for 20 rounds.

The server holds the shared model. In each round it chooses 4 of the 8 clients, sends them the current model, receives their updates and averages them, weighting each client by how many photos it has (FedAvg). After each round the new shared model is scored on the validation set; the test set is used once at the end.

5.3 Data split between clients

The training photos are divided among 8 simulated clients with a **Dirichlet distribution (alpha = 0.5)**. A small alpha makes each client's class mix uneven, which mimics real farms where one disease dominates. Each client keeps at least 20 photos. All four experiments use the same split and the same random seed (42), so differences between them come from the method, not from luck in the data.

5.4 Client selection

- **Random:** choose 4 of 8 clients uniformly at random each round.
- **Adaptive:** score each client as 0.5 x (its last training loss, scaled 0 to 1) + 0.3 x (rounds since it was last chosen, scaled) + 0.2 x (its data size, scaled), then pick the 4 highest. A client that has never been chosen counts as having the maximum loss, so every client gets a turn early. Ties go to the lower client number.

5.5 Top-k compression with error feedback

After local training, a compressed client keeps only the largest 10% of its update entries (by absolute value) and sends their positions (4 bytes each) and values (4 bytes each): 8 bytes per kept entry instead of 4 bytes per entry for a full update. The entries it did not send are stored on the client as a *residual* and added to the next update, so they are delayed rather than lost. The server rebuilds a full-size update by filling the missing entries with zeros.

5.6 What is measured

- **Accuracy and macro-F1** of the shared model on validation data after every round, and on the test set once at the end. Macro-F1 averages the F1 score of all 16 classes, so rare classes count as much as common ones.
- **Communication in bytes.** Downlink (server to client) is the full model times the number of selected clients. Uplink (client to server) is the real size of the arrays returned. Top-k reduces uplink only; the downlink stays full size.

- **Convergence.** The first round whose validation accuracy is within 0.01 of the run's best. The comparison script also reports rounds to a common target (95% of the lowest best accuracy among the runs), so runs are compared fairly.
- **Per-class and per-client results** and a confusion matrix (which diseases get mistaken for which).

5.7 The four experiments

Experiment	Client selection	Update compression	Question it answers
A: FedAvg baseline	Random	None	The reference every other run is compared with
B	Adaptive	None	Does smarter selection help on its own?
C	Random	Top-k (10%)	Does compression help on its own?
D	Adaptive	Top-k (10%)	Do the two ideas help together?

Shared settings: 8 clients, 4 chosen per round, 20 rounds, 1 local epoch, learning rate 0.01, batch size 32, images resized to 160 px, seed 42.

6. Web application

The website is built with Next.js and works in light and dark themes. Pages inside the application require sign-in; the landing, login and register pages are public.

Page	Purpose
Landing	Explains the project, a four-step federated round, features and dataset credit
Login and Register	Email and password sign-in with validation and readable error messages
Dashboard	Summary of experiments imported for the signed-in user
Experiments and experiment detail	List of runs; detail page shows rounds, per-class metrics (rare classes flagged), per-client results and a shaded confusion matrix
Results	Compare chosen experiments: accuracy, macro-F1, training and validation loss, final precision, recall and F1, cumulative communication, client participation. Every chart has a table view and each experiment keeps one fixed colour
Diagnosis	Upload a leaf photo, see the predicted disease, confidence, top three guesses, a plain-language explanation and a guide to all 16 classes
About	Problem, objectives, workflow, selection method, Top-k, the A to D table and limitations
404 and error pages	Friendly fallbacks

6.1 Honesty rules built into the interface

- Missing values are shown as an em dash or "No data yet", never as zero.
- With no results imported, the pages say so; no placeholder numbers are shown.
- When no prediction service is connected, the diagnosis page shows a clearly labelled placeholder and disables the button; it never invents a prediction.
- Every result carries the notice that it is a model estimate and not a professional diagnosis, and results below 50% confidence get an extra low-confidence warning.

6.2 Data model and security

Each signed-in user owns a private area of the database: experiments with their per-round metrics, and a history of predictions. The security rules, enforced by the database itself and not only by the website, allow each user to read only their own data, deny every client write to experiments and rounds (only the trusted Python import script may write them, using an administrator credential kept outside the code), allow a user to create a prediction record only if it has the expected fields and a server timestamp, and deny everything else by default. Ten automated tests check these rules against the Firebase emulator.

Privacy. A prediction record stores only the predicted label, the confidence, the model version and a timestamp. The photo itself is never stored: it is held in the browser, sent to the prediction service in memory and discarded. No passwords or keys appear in the source code; real configuration lives in an ignored `.env.local` file, and an `.env.example` shows the format.

6.3 Plain-language disease guide

Many class names are abbreviations (TSWV, TMV, PVY, CMV). The result card therefore shows the full name, the cause (fungus, virus, bacterium, water mould, pest, environment or genetic), the organism where one exists, a one-line summary and what the disease usually looks like. A guide on the same page describes all 16 classes. Several virus names mention another crop (for example *tomato* spotted wilt virus) only because it was first described there; they also infect tobacco. These descriptions were written from general plant-pathology knowledge and have **not yet been reviewed by an expert**; they are background, not treatment advice.

7. Prediction service

The prediction service is a small FastAPI program that loads the trained model and answers one question: given a photo, which class is it? The website sends the photo as a form upload and receives the label, the confidence, the three most likely classes and the model version.

- **Input checks.** Only JPEG, PNG or WebP; files over 5 MB, empty or corrupt files are rejected with clear error codes (415, 413, 400). Cross-origin access is limited to the configured website addresses.
- **Preparation.** The photo is colour-balanced with the Shades-of-Gray method (power 6) and resized to the model's size. This is an approximation of the dataset authors' processing, whose exact settings were not published. A check on the test photos showed the extra step changes accuracy by about 1.5 points (84.1% versus 82.6%); it is not the main source of error on outside photos.
- **Safety.** The service refuses to start without a trained model, so it can never answer from random weights. Its automated tests use a throwaway random model, and they check behaviour (types, limits, response shape), not accuracy.
- **Where it runs.** On the developer's computer. A deployed website cannot reach it until it is hosted somewhere; this is an open decision (section 14).

8. Verification and testing

The table lists what was actually checked, and how. Items marked as not verified should not be assumed to work.

Area	Check	Result
Security rules	10 tests on the Firebase emulator (owner reads, denied writes, validated prediction create, default deny)	Pass
Results import	Python import into the emulator, then the web app's own parser reads it back (2 tests)	Pass, with smoke data
Python pipeline	20 tests: partitioning, Top-k round trip, selectors, metrics, strong augmentation, prediction service	Pass
Web code	ESLint and TypeScript type checks	Pass at the last run
Web pages	All routes returned HTTP 200 in the dev server; unknown route returned 404	Pass
Prediction service	Real uploads of test photos through the HTTP service	Works; accuracy measured separately
Federated runs	Smoke runs of A, B and D (tiny settings, about 13% accuracy)	Run only; results meaningless
Experiment C	Its own smoke run	**Not done**
GPU path for federated runs	Resource settings exist in code	**Not verified**
Charts with real data	Rendering of real federated results	**Not verified**
Visual review by a person	Reading every page in a browser	**Not done**

8.1 Pipeline smoke test (not a result)

To prove the pipeline runs end to end, experiments A, B and D were run for a handful of rounds on tiny settings. Their accuracy (about 13%) is meaningless. One observation is worth recording as a design fact, not a finding: in the A-versus-D comparison of that run, Top-k cut the upload from 36.5 MB to 7.3 MB (5 times smaller), but total traffic only fell to 0.60 of the baseline because the download from server to client stays full size. The real experiments will show whether this holds.

9. Results: the centralised baseline

The federated experiments have not been run, so there are no federated results. The only real results are for a conventional classifier trained on all training photos in one place. It serves two purposes: it provides the model behind the website's diagnosis page, and it gives an upper reference that federated runs are expected to approach.

9.1 Two versions of the baseline

The first baseline used basic training (constant learning rate 0.01, 160 px images, 15 epochs). Its validation score swung widely from epoch to epoch (Figure 4), a sign that training was unstable. An improved recipe was then tried on Google Colab with a GPU. Four variants were compared on **validation data only**, and the test set was evaluated once, on the 224 px variant.



Figure 4. Validation macro-F1 per epoch. The dot marks the epoch whose weights were saved (highest validation score). Both curves are read from the saved training logs.

Validation comparison of the four variants (30 epochs each)

Variant	What was added	Best val macro-F1	Mean of last 5 epochs	Best val accuracy
r1	Lower learning rate with cosine decay	0.822	0.741	0.928
r2	r1 + stronger augmentation + label smoothing	0.781	0.761	0.891
r3	r2 + class-weighted loss	0.862	0.813	0.906
r4 (tested)	r3 at 224 px instead of 160 px	0.840	0.795	0.920

Values as printed by the Colab comparison step. The last-5 average shows how stable training is. Differences of a few hundredths between variants are within the noise of a 138-photo validation set.

An honest note on selection. The test run used the 224 px variant (r4). The 160 px variant with class weighting (r3) scored slightly higher on validation (0.862 versus 0.840) but was not tested; testing both and picking the better one by test score would have used the test set for selection. Also, the saved r4 weights are from epoch 6, the highest validation score of the run (0.840). Later epochs settled between 0.78 and 0.81, so that validation figure is probably a slightly lucky peak. Finally, the first baseline's test score was seen before the improvements were designed.

9.2 Test-set results (138 photos)

Model	Accuracy	Macro precision	Macro recall	Macro-F1
First baseline (160 px, basic training)	76.8%	0.818	0.823	0.791
Improved baseline (224 px, tuned training)	84.1%	0.732	0.756	0.729

How to read these numbers. Accuracy rose from 76.8% to 84.1%, but macro-F1 fell from 0.791 to 0.729. Both are real and they do not contradict each other. Macro-F1 gives each of the 16 classes equal weight, and four classes have a single test photo. The improved model missed two of those single photos (anthracnose and tomato spotted wilt virus), each scoring 0 and together costing about 0.12 of macro-F1. With 138 photos, an accuracy of 84.1% carries roughly +/-6 points of uncertainty (95% interval), and the earlier 76.8% roughly +/-7. The two accuracies therefore overlap somewhat; the gain is likely real but its size is uncertain.

9.3 Results by disease

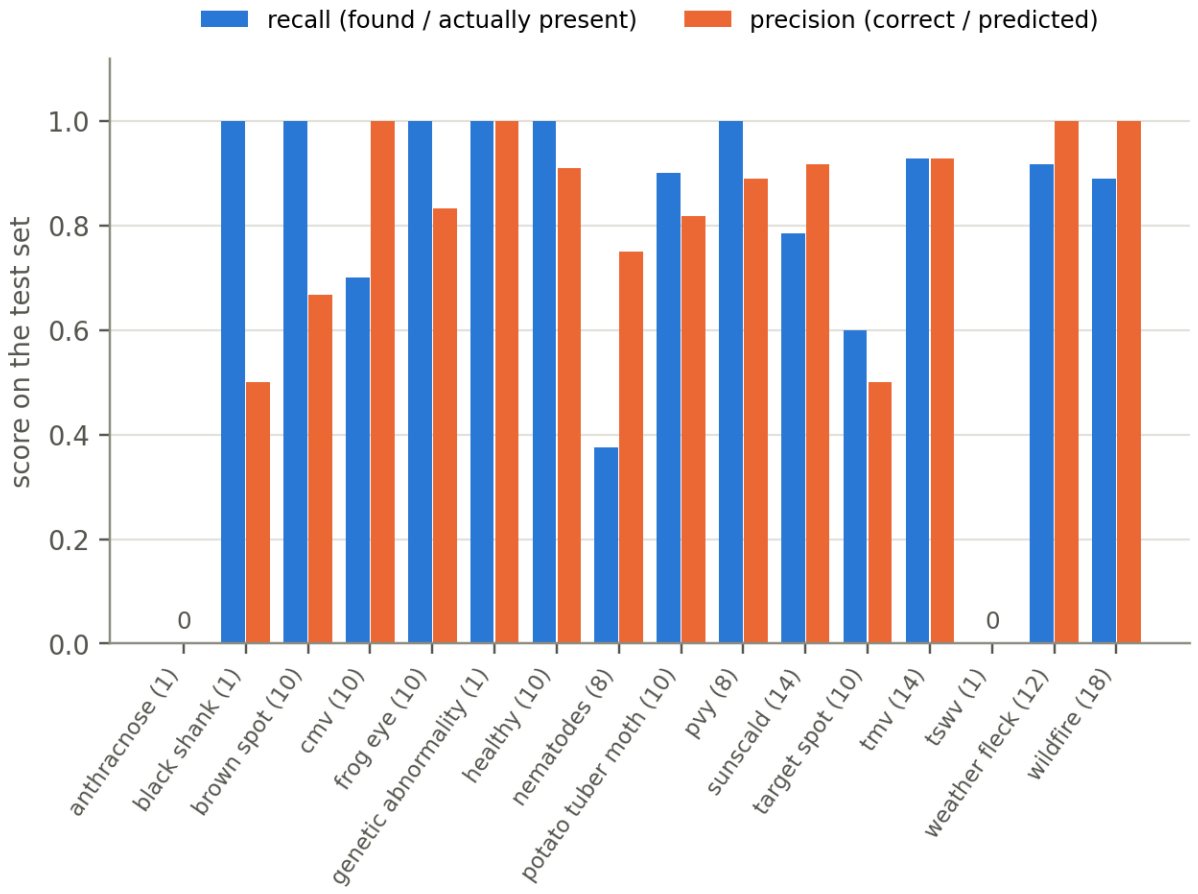


Figure 5. Test-set recall and precision of the improved baseline per class. The number in brackets is the class's test photo count. Anthracnose and TSWV scored 0 (one photo each).

Class	Test photos	Precision	Recall	F1
anthracnose	1	0.00	0.00	0.00
black shank	1	0.50	1.00	0.67
brown spot	10	0.67	1.00	0.80
cmv	10	1.00	0.70	0.82
frog eye	10	0.83	1.00	0.91
genetic abnormality	1	1.00	1.00	1.00
healthy	10	0.91	1.00	0.95
nematodes	8	0.75	0.38	0.50
potato tuber moth	10	0.82	0.90	0.86
pvy	8	0.89	1.00	0.94
sunscald	14	0.92	0.79	0.85
target spot	10	0.50	0.60	0.55
tmv	14	0.93	0.93	0.93
tswv	1	0.00	0.00	0.00
weather fleck	12	1.00	0.92	0.96
wildfire	18	1.00	0.89	0.94

- **Strongest:** pvy, frog eye, brown spot, healthy and genetic abnormality all reach recall 1.0 (genetic abnormality on a single photo).
- **Weakest with several photos:** nematodes (recall 0.38), target spot (0.60) and sunscald (0.79).
- **Over-predicted:** brown spot, black shank and target spot have precision of 0.67, 0.50 and 0.50, so other diseases are sometimes labelled as these.

10. Reliability: what the numbers can and cannot support

- **Small test set.** 138 photos. Sixteen classes, four of them with one test photo. One photo moves a rare class from 0 to 1 and macro-F1 by about 0.06.
- **One split.** All scores come from one training, validation and test split. Repeated cross-validation would give a much more trustworthy estimate and error bar. It has not been done.
- **Selection on a small validation set.** Choosing the best epoch and the best variant from the same 138-photo validation set makes validation figures optimistic. That is why the test set is the headline.
- **Confidence is not trustworthy as a guide to correctness.** A wrong answer can carry 50% to 90% confidence, so a confidence threshold cannot reliably catch errors.
- **The model always answers.** It picks one of 16 classes even for something that is not a tobacco leaf.

11. Limitations

11.1 Photos from other sources

This is the most important limitation found so far. Three photos supplied by the project owner from other sources were classified wrongly by the improved model:

Photo label (as given)	Meaning	Model answer	Confidence
Early_FLS	Early frogeye leaf spot (class frog eye)	Weather fleck	40.5%
cercni	Cercospora nicotianae, frogeye (class frog eye)	Weather fleck	54.6%
weather fleck	Weather fleck	Genetic abnormality (weather fleck second, 25.5%)	52.8%

Three photos are far too few to measure accuracy and the labels were given by the owner, not independently verified. They do show the model failing on every outside photo tried, in contrast to about 83% to 84% on the dataset's own test photos. The likely explanation is that outside photos look different from the training photos (camera, lighting, close-up crops, early-stage lesions) but this cause is not proven. A first suspect, the prediction service's colour correction, was ruled out as the main cause. Until a larger outside test is done, the honest claim is: **the model works on photos like those in its dataset; behaviour on other photos is unknown and so far poor.**

11.2 Other limitations

- **Federated results will be simulated.** All clients run on one machine. Communication is measured in bytes sent, not network time, and there are no real network faults or device limits.
- **Rare classes.** Four classes have 7 to 18 real training photos. Whatever a federated or centralised model learns for them is fragile.
- **Dataset licence.** No explicit licence is stated; academic use with citation is assumed and the data cannot be redistributed.
- **Not a diagnosis.** The output is an estimate from photographs. Real crop problems need an agronomist.
- **Disease descriptions are unreviewed.** They are general background and have not been checked by a plant-pathology expert.

- **Service is local.** The prediction service runs on a single PC. A hosted demo would need a place to run it (and possibly a cost).
- **Training cost.** Real federated runs on a laptop CPU would take hours; they need a GPU. The GPU path for these runs has not been tested.
- **No version control yet.** Nothing is committed to git.

12. Completion status

The table lists each work item, its state and what is left. States: **Done** means built and checked as described in section 8; **Partly** means some of it exists; **Not started** means no work yet.

#	Work item	State	What is left
1	Dataset audit, split and leakage controls	Done	Optional: repeated cross-validation for rare classes
2	Web application pages and design	Done	Visual review in a browser by a second person
3	Authentication and Firestore rules	Done	Confirm rules are published to the live Firebase project
4	Federated-learning code and tests	Done	Smoke run of experiment C; GPU path check
5	Export, compare and import tools	Done	Run with real results
6	Centralised baseline model	Done	Optional: test the r3 variant and report both
7	Prediction service and diagnosis page	Done	Host the service for a public demo
8	Federated experiments A to D	Not started	Run on a GPU, then compare
9	Real results shown in the web app	Not started	Import, then check charts and tables
10	Tests on photos from other sources	Not started	Collect labelled outside photos and measure
11	Deployment	Not started	Needs owner's approval; Vercel account
12	Final report and version control	Partly	Add federated results; commit the code

Reading the status. 7 items are done, 1 is partly done and 4 are not started. The items not started include the core research experiments and the check on outside photos. The project is complete as software, but not yet complete as research.

13. Future scope and remaining work

13.1 Required to complete the project (in order)

- **Run experiments A to D** on a Colab GPU with the same improved training settings, so the four runs stay comparable. Run each at least once with the shared seed; multiple seeds would show how much results vary.
- **Compare and import.** Produce the comparison table, then import results into Firebase using the owner's account and a service-account key stored outside the repository.
- **Check the website with real results.** Charts, tables, the confusion matrix and the client-participation view have only been seen with smoke data.
- **Test on outside photos.** Collect a few dozen labelled photos from other sources, measure per-class accuracy and decide whether retraining on more varied data is needed.
- **Expert review** of the disease descriptions and class names.
- **Deploy the website** to Vercel after approval, add its address to Firebase's authorised domains, and decide where the prediction service runs.
- **Version control and final write-up** including limitations and dataset credits.

13.2 Improvements that would strengthen the work

- **More reliable scoring.** Repeated stratified cross-validation, reported with error bars, instead of a single 138-photo test set.
- **More data for rare classes** and for outside conditions. The Roboflow frogeye and brown-spot images (CC BY 4.0) could be added with proper credit, split by original photo so duplicate or augmented copies never straddle training and test.
- **A "not a tobacco leaf / unsure" outcome** so the model can decline to answer, and better-calibrated confidence.
- **Other federated methods** to compare against, such as methods designed for uneven client data, and other compressors such as quantisation.
- **Privacy extras** such as secure aggregation or differential privacy, which federated learning alone does not provide.
- **Real-device trials** with phones or edge devices to measure true network and energy cost.
- **Usability work** such as local-language text, a mobile-friendly capture guide for good photos, and feedback from growers.
- **Hosting and monitoring** of the prediction service, including usage limits.

These are suggestions, not commitments; none has been started.

14. How to run and reproduce

These commands are from the project's own scripts. Detailed set-up is in the repository's README files and the `docs` folder.

Task	Command
Website (from the web folder)	`npm run dev`
Rule tests and import test	`npm run test:rules and npm run test:import`
Python tests (from the ml folder)	`python -m pytest`
Train the centralised baseline	`python -m src.training.centralised --epochs 30 --lr 0.005 --cosine --strong-aug --label-smoothing 0.1 --balance --img-size 224 --skip-test --out <name>`
Test a chosen run once	`python -m src.training.final_test --out <name>`
Run all four federated experiments	`python -m src.federated.run_all`
Compare runs	`python -m src.export.compare`
Import results into Firebase	`python -m src.export.import_to_firestore (with the owner's user id and service-account key; --dry-run to preview)`
Start the prediction service	`set MODEL_DIR=<name> then python -m src.serve.app`

Configuration lives in `ml/configs/base.yaml`, which drives all four experiments so they share data, model and training settings. A Colab notebook (`ml/colab/train_on_colab.ipynb`) runs the baseline trials on a GPU. Secrets are never committed: `.env*` files, service-account keys and model files are ignored by git.

Conventions followed

- No metrics, predictions or dataset statistics were invented; every number in this report is read from saved results or stated as given.
- No training code in the website, and none on the hosting platform.
- Authorisation is enforced by database rules, not only by the interface.
- Anything that was not tested is labelled as such.

15. References and credits

- Lin et al., Tobacco Leaf Abnormality dataset papers: SPIE International Conference on Digital Image Processing (ICDIP) 2022, and Frontiers in Plant Science 2024. Dataset: Hugging Face `TamAko783/TPDD_Honglin_CLS`.
- McMahan, Moore, Ramage, Hampson and Aguera y Arcas, *Communication-Efficient Learning of Deep Networks from Decentralized Data*, AISTATS 2017 (FedAvg).
- Sandler, Howard, Zhu, Zhmoginov and Chen, *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, CVPR 2018.
- Beutel et al., *Flower: A Friendly Federated Learning Framework*, 2020 (arXiv:2007.14390).
- Lin, Han, Mao, Wang and Dally, *Deep Gradient Compression*, ICLR 2018; Stich, Cordonnier and Jaggi, *Sparsified SGD with Memory*, NeurIPS 2018 (Top-k with error feedback).
- Finlayson and Trezzi, *Shades of Gray and Colour Constancy*, Color and Imaging Conference, 2004.
- Kunyadini, *Tobacco leaf disease detection*, Roboflow Universe, CC BY 4.0 (considered, not used for training).

Reference details are given from memory of the standard citations and should be checked against the originals before submission.

Appendix A. Glossary

Term	Plain meaning
Federated learning	Training a shared model while the training data stays with its owners; only model updates travel
FedAvg	The standard way to combine client updates: a weighted average, weighted by each client's number of photos
Client / round	A data owner taking part (here a simulated farm); one cycle of select, train, upload, combine
Non-IID	Clients hold different kinds of data (for example different diseases), not a random mix
Top-k compression	Sending only the k largest entries of an update, with their positions
Error feedback	Remembering the entries not sent and adding them to the next update
Gradient / update	The change a client makes to the model during local training
Accuracy	Share of photos classified correctly
Precision	Of the photos the model labelled as class X, the share that really are X
Recall	Of the photos that really are X, the share the model found
F1 and macro-F1	F1 balances precision and recall for one class; macro-F1 averages F1 over all classes equally
Confusion matrix	A table of true class against predicted class that shows which diseases are mixed up
Validation vs test set	Validation guides choices during development; the test set is kept aside for one final score
Leakage	When near-copies of training photos appear in test data, making scores look better than they are
Overfitting	A model memorising its training photos instead of learning general patterns
Epoch	One full pass over the training photos
Smoke test	A tiny run that only proves the pipeline works; its numbers mean nothing
Shades-of-Gray	A colour-balancing method that removes the colour cast of lighting

Appendix B. The 16 classes in plain language

General background written for readers who are not plant-pathology specialists. These descriptions have not been reviewed by an expert and are not treatment advice.

Class (model label)	Cause	In brief
Healthy leaf (healthy)	None	No disease or damage; even green colour
Anthracnose (anthracnose)	Fungus (Colletotrichum)	Sunken or dry brown to tan lesions, sometimes with darker edges
Black shank (black_shank)	Water mould (Phytophthora nicotianae)	Soil-borne; yellowing and wilting, stem base turns black
Brown spot (brown_spot)	Fungus (Alternaria)	Round brown spots, often with target-like rings, mostly on older leaves
Cucumber mosaic virus (cmv)	Virus	Light and dark green mottling, puckered or narrow leaves; spread by aphids
Frogeye leaf spot (frog_eye)	Fungus (Cercospora nicotianae)	Round spots with a pale centre and dark border
Genetic abnormality (genetic_abnormality)	Genetic	Unusual leaf shape, colour or pattern not caused by a pathogen
Nematode damage (nematodes)	Pest (root-knot nematodes)	Stunting and patchy yellowing; the roots are affected
Potato tuber moth damage (potato_tuber_moth)	Pest (caterpillar)	Winding tunnels or transparent blotches inside the leaf
Potato virus Y (pvy)	Virus	Mottling, dark or dead veins; spread by aphids
Sunscald (sunscald)	Environment	Bleached, papery patches from intense sun
Target spot (target_spot)	Fungus (Rhizoctonia solani)	Brown spots with rings; the dead centre may fall out leaving holes
Tobacco mosaic virus (tmv)	Virus	Mosaic pattern, curled or distorted leaves; spreads by touch and tools
Tomato spotted wilt virus (tswv)	Virus	Brown or bronze ring spots and dead patches; spread by thrips
Weather fleck (weather_fleck)	Environment (ozone)	Many small white, tan or grey flecks on upper sides of older leaves
Wildfire (wildfire)	Bacterium (Pseudomonas syringae pv. tabaci)	Brown spots with a wide yellow halo, spreads in wet windy weather